

# Decision Tree – Complete Assignment Answers

Question 1: What is a Decision Tree, and how does it work in the context of classification?

Answer:

A **Decision Tree** is a supervised Machine Learning algorithm used for both **classification and regression** tasks. In classification, it predicts the class or category of a target variable by making a sequence of decisions based on the input features.

A Decision Tree looks like a flowchart:

- **Root Node:** The first and most important decision.
- **Internal Nodes:** Conditions based on features.
- **Branches:** Possible outcomes of decisions.
- **Leaf Nodes:** Final predicted classes.

How it works in Classification:

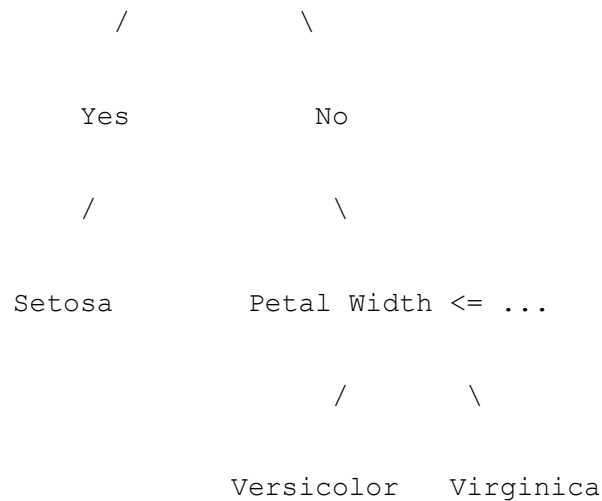
1. Start with the complete dataset at the root node.
2. Select the best feature and threshold for splitting the data.
3. Divide the data into smaller groups.
4. Continue splitting the groups until a stopping condition is reached.
5. The final leaf node gives the predicted class.

Example:

Suppose we want to predict whether a flower is **Setosa, Versicolor, or Virginica**.

The tree may make decisions such as:

Petal Length  $\leq$  2.45?



Easy Definition:

**A Decision Tree makes a series of simple decisions to reach a final prediction.**

---

Question 2: Explain the concepts of Gini Impurity and Entropy as impurity measures. How do they impact the splits in a Decision Tree?

Answer:

**Gini Impurity** and **Entropy** are measures used by Decision Trees to determine how pure or impure a node is.

A **pure node** contains observations belonging mostly or completely to one class.

## 1. Gini Impurity

Gini Impurity measures the probability of incorrectly classifying a randomly selected observation if it were randomly labeled according to the class distribution.

Formula:

$$\text{Gini} = 1 - \sum_{i=1}^C p_i^2$$

Where:

- $(p_i)$  = proportion of observations belonging to class  $i$
- $C$  = number of classes

For a binary classification with 50% observations in each class:

$$\text{Gini} = 1 - \left( 0.5^2 + 0.5^2 \right) = 0.5$$

A lower Gini value indicates a purer node.

---

## 2. Entropy

Entropy measures the level of uncertainty or disorder in a node.

Formula:

Entropy=

$$-\sum_{i=1}^C p_i \log_2(p_i)$$

A pure node has:

Entropy=0

Higher entropy means greater impurity.

Impact on Decision Tree Splits:

The Decision Tree evaluates possible splits and selects the split that produces the **purest child nodes**.

For Gini:

**Lower weighted Gini impurity = Better split**

For Entropy:

**Higher Information Gain = Better split**

Comparison:

| Measure       | Main Idea            | Best Split               |
|---------------|----------------------|--------------------------|
| Gini Impurity | Measures impurity    | Lowest impurity          |
| Entropy       | Measures uncertainty | Highest information gain |

Easy Definition:

**Gini and Entropy help the Decision Tree decide where to split the data.**

---

Question 3: What is the difference between Pre-Pruning and Post-Pruning in Decision Trees?  
Give one practical advantage of using each.

Answer:

**Pruning** is the process of controlling the size and complexity of a Decision Tree to reduce overfitting.

There are two main types:

## 1. Pre-Pruning

Pre-Pruning stops the tree from growing too much during the training process.

Common parameters include:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`

- `max_leaf_nodes`

### Practical Advantage:

It **reduces training time and prevents the tree from becoming unnecessarily complex.**

---

## 2. Post-Pruning

Post-Pruning allows the tree to grow first and then removes branches that do not provide sufficient predictive value.

In Scikit-learn, cost-complexity pruning can be performed using:

`ccp_alpha`

### Practical Advantage:

It can **remove unnecessary branches after seeing the complete tree structure**, often improving generalization.

### Difference:

| Pre-Pruning               | Post-Pruning                           |
|---------------------------|----------------------------------------|
| Stops tree growth early.  | Grows tree and removes branches later. |
| Applied during training.  | Applied after tree construction.       |
| Can reduce training time. | Can simplify an already-grown tree.    |

Uses parameters such as `max_depth`.

Can use `ccp_alpha`.

Easy Definition:

**Pre-Pruning prevents excessive growth; Post-Pruning removes unnecessary growth.**

---

Question 4: What is Information Gain in Decision Trees, and why is it important for choosing the best split?

Answer:

**Information Gain** measures how much uncertainty or impurity is reduced after splitting a dataset.

It is mainly associated with **Entropy**.

Formula:

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$$

A good split produces child nodes that are more pure than the parent node.

Example:

Suppose the parent node has high entropy.

After splitting:

- Left child has mostly Class A.
- Right child has mostly Class B.

The entropy decreases significantly.

Therefore, the split has **high Information Gain**.

## Why is Information Gain Important?

Decision Trees may have many possible features and split points. Information Gain helps the algorithm choose the split that gives the **largest reduction in uncertainty**.

Easy Definition:

**Information Gain tells us how useful a split is for separating different classes.**

---

## Question 5: What are some common real-world applications of Decision Trees, and what are their main advantages and limitations?

Answer:

Decision Trees are widely used in many real-world applications.

## Real-World Applications

### 1. Healthcare

Predict whether a patient is likely to have a disease based on medical information.

### 2. Banking

Predict whether a customer is likely to default on a loan.

### 3. Marketing

Predict whether a customer will purchase a product.

### 4. Insurance

Estimate insurance risk and classify customers into risk categories.

### 5. Human Resources

Predict employee attrition or classify candidates.

### 6. E-commerce

Predict customer purchase behavior and customer segments.

---

## Advantages of Decision Trees

1. **Easy to understand and interpret**
  2. Can handle both numerical and categorical features.
  3. Requires relatively little data preprocessing.
  4. Can model non-linear relationships.
  5. Feature importance can be extracted.
  6. Useful for both classification and regression.
- 

## Limitations of Decision Trees

1. Can easily **overfit** the training data.
2. Small changes in data can produce a different tree.
3. Very deep trees can become difficult to interpret.

4. A single tree may have lower predictive performance than ensemble methods such as Random Forest.
5. Trees can sometimes favor features with many possible split points.

## Summary:

| Advantages                       | Limitations                      |
|----------------------------------|----------------------------------|
| Easy to interpret                | Can overfit                      |
| Handles non-linear relationships | Sensitive to data changes        |
| Little preprocessing             | Deep trees become complex        |
| Feature importance available     | Single tree may be less accurate |
| Classification + Regression      | Can produce unstable trees       |

---

## Dataset Information

### Iris Dataset

The **Iris Dataset** is commonly used for classification.

It contains measurements of flowers such as:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Target classes:

- Setosa
- Versicolor

- Virginica

It can be loaded using:

```
from sklearn.datasets import load_iris
```

---

**Question 6: Write a Python program to Load the Iris Dataset, Train a Decision Tree Classifier using the Gini Criterion, and Print Accuracy and Feature Importances.**

**Answer:**

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score


# Load Iris Dataset

iris = load_iris()
```

```
X = iris.data

y = iris.target


# Split the dataset

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y

)


# Create Decision Tree Classifier using Gini criterion

model = DecisionTreeClassifier(

    criterion="gini",

    random_state=42

)


# Train the model

model.fit(X_train, y_train)


# Make predictions

y_pred = model.predict(X_test)
```

```
# Calculate accuracy

accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)

# Print feature importances

print("\nFeature Importances:")

for feature, importance in zip(

    iris.feature_names,

    model.feature_importances_

):

    print(f"{feature}: {importance:.4f}")
```

## Explanation:

- `criterion="gini"` tells the model to use Gini Impurity.
- `train_test_split()` divides the data into training and testing sets.
- `model.fit()` trains the Decision Tree.
- `model.predict()` generates predictions.
- `accuracy_score()` calculates classification accuracy.

- `feature_importances_` shows the relative importance of each feature.

### Observation:

The feature importance values indicate which Iris features were most useful for making classification decisions.

---

Question 7: Write a Python program to Load the Iris Dataset, Train a Decision Tree Classifier with `max_depth=3` and Compare its Accuracy to a Fully-Grown Tree.

### Answer:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score


# Load dataset

iris = load_iris()
```

```
X = iris.data

y = iris.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y

)


# Fully-grown Decision Tree

full_tree = DecisionTreeClassifier(

    criterion="gini",

    random_state=42

)


full_tree.fit(X_train, y_train)


# Predictions

full_pred = full_tree.predict(X_test)
```

```
# Accuracy

full_accuracy = accuracy_score(y_test, full_pred)


# Decision Tree with max_depth=3

limited_tree = DecisionTreeClassifier(

    criterion="gini",

    max_depth=3,

    random_state=42

)


limited_tree.fit(X_train, y_train)


# Predictions

limited_pred = limited_tree.predict(X_test)


# Accuracy

limited_accuracy = accuracy_score(y_test, limited_pred)


# Print results
```

```
print("Fully-Grown Tree Accuracy:", full_accuracy)

print("max_depth=3 Tree Accuracy:", limited_accuracy)
```

## Comparison:

The fully-grown tree is allowed to continue splitting until its stopping conditions are reached.

The `max_depth=3` tree is restricted to a maximum depth of three levels.

## Observation:

A shallower tree is generally less complex and may reduce overfitting. The best choice should be based on validation/test performance rather than assuming that a deeper or shallower tree is always better.

---

# Question 8: Write a Python Program to Load the Boston Housing Dataset, Train a Decision Tree Regressor, and Print MSE and Feature Importances.

## Important Note:

`load_boston()` has been removed from recent versions of Scikit-learn because the Boston Housing dataset has ethical and methodological concerns.

Therefore, in modern Google Colab environments, use the **provided Boston Housing CSV** if your assignment provides one.

If you have a CSV file, use the following code.

## Python Program:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import mean_squared_error


# Load Boston Housing CSV

df = pd.read_csv("Boston_Housing.csv")


# Display columns

print(df.columns)


# Replace 'MEDV' with the actual target column name if different

X = df.drop("MEDV", axis=1)

y = df["MEDV"]


# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
# Create Decision Tree Regressor
```

```
model = DecisionTreeRegressor(
```

```
    random_state=42
```

```
)
```

```
# Train model
```

```
model.fit(X_train, y_train)
```

```
# Predictions
```

```
y_pred = model.predict(X_test)
```

```
# Calculate MSE
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
print("Mean Squared Error (MSE):", mse)
```

```
# Feature Importances

print("\nFeature Importances:")

for feature, importance in zip(

    X.columns,

    model.feature_importances_

):

    print(f"{feature}: {importance:.4f}")
```

## Explanation:

- `DecisionTreeRegressor` is used for continuous target prediction.
- `mean_squared_error()` calculates the MSE.
- `feature_importances_` identifies features that contributed most to the predictions.

## Note:

If your CSV uses another target column name, replace "MEDV" with the correct target column.

---

## Question 9: Write a Python Program to Tune max\_depth and min\_samples\_split Using GridSearchCV.

Answer:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score


# Load Iris dataset

iris = load_iris()


X = iris.data

y = iris.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y
```

```
)
```

```
# Create Decision Tree
```

```
tree = DecisionTreeClassifier(
```

```
    criterion="gini",
```

```
    random_state=42
```

```
)
```

```
# Define hyperparameter grid
```

```
param_grid = {
```

```
    "max_depth": [2, 3, 4, 5, None],
```

```
    "min_samples_split": [2, 5, 10]
```

```
}
```

```
# GridSearchCV
```

```
grid_search = GridSearchCV(
```

```
    estimator=tree,
```

```
    param_grid=param_grid,
```

```
    cv=5,
```

```
        scoring="accuracy"

    )

    # Train GridSearchCV

    grid_search.fit(X_train, y_train)

    # Best parameters

    print("Best Parameters:")

    print(grid_search.best_params_)

    # Best model

    best_model = grid_search.best_estimator_

    # Test predictions

    y_pred = best_model.predict(X_test)

    # Test accuracy

    accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nTest Accuracy:", accuracy)
```

## Explanation:

GridSearchCV automatically tests different combinations of hyperparameters.

In this program:

- `max_depth` controls the maximum depth of the tree.
- `min_samples_split` specifies the minimum number of samples required to split an internal node.
- `cv=5` performs 5-fold cross-validation.
- `best_params_` returns the best parameter combination.

## Advantage:

GridSearchCV helps select hyperparameters systematically instead of choosing them manually.

---

# Question 10: Healthcare Decision Tree – Complete Step-by-Step Process

## Scenario:

A healthcare company wants to predict whether a patient has a particular disease. The dataset contains numerical and categorical variables and also has missing values.

The complete Machine Learning process would be:

---

# Step 1: Understand the Business Problem

First, clearly define the objective.

Target Variable:

Disease

For example:

0 = No Disease

1 = Disease

The goal is to predict whether a patient belongs to the disease or non-disease class.

---

## Step 2: Explore the Dataset

First inspect the dataset.

```
df.head()
```

```
df.info()
```

```
df.describe()
```

```
df.isnull().sum()
```

Check:

- Number of rows and columns
  - Data types
  - Missing values
  - Duplicate records
  - Target distribution
  - Numerical and categorical features
- 

## Step 3: Handle Missing Values

Missing values should be handled before training the model.

### Numerical Features

Use methods such as:

- Median imputation
- Mean imputation

Median is often useful when the data contains outliers.

Example:

```
df["age"] = df["age"].fillna(df["age"].median())
```

### Categorical Features

Use the most frequent category or another suitable category.

```
df["gender"] = df["gender"].fillna(  
  
    df["gender"].mode()[0]  
  
)
```

For a production project, the imputation values should be learned from the training data only to avoid data leakage.

---

## Step 4: Encode Categorical Features

Machine Learning models generally require numerical inputs.

Categorical variables can be converted into numerical form.

### One-Hot Encoding

For example:

Gender:

Male

Female

After encoding:

Gender\_Male

Gender\_Female

Python:

```
df = pd.get_dummies(
```

```
df,  
  
columns=["gender"],  
  
drop_first=True  
)
```

For a robust train/test workflow, `OneHotEncoder` inside a Scikit-learn `Pipeline` or `ColumnTransformer` is preferable.

---

## Step 5: Separate Features and Target

```
X = df.drop("Disease", axis=1)  
  
y = df["Disease"]
```

Where:

- `X` = input features
  - `y` = target variable
- 

## Step 6: Split the Dataset

Divide the data into training and testing sets.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
  
    X,  
  
    y,  
  
    test_size=0.2,  
  
    random_state=42,  
  
    stratify=y  
)
```

Here:

- 80% data → Training
- 20% data → Testing

`stratify=y` helps maintain a similar class distribution in the train and test sets.

---

## Step 7: Train the Decision Tree

Create and train the classifier.

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier()
```

```
        criterion="gini",

        random_state=42

    )

model.fit(X_train, y_train)
```

---

## Step 8: Tune Hyperparameters

Decision Trees can overfit, so hyperparameters should be tuned.

Important parameters include:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `max_leaf_nodes`
- `criterion`
- `ccp_alpha`

### GridSearchCV Example:

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {

    "max_depth": [3, 5, 7, 10, None],
```

```
    "min_samples_split": [2, 5, 10],  
  
    "min_samples_leaf": [1, 2, 5]  
}
```

```
grid_search = GridSearchCV(  
  
    DecisionTreeClassifier(  
  
        criterion="gini",  
  
        random_state=42  
    ),  
  
    param_grid,  
  
    cv=5,  
  
    scoring="f1"  
)
```

```
grid_search.fit(X_train, y_train)
```

```
print("Best Parameters:")
```

```
print(grid_search.best_params_)
```

## Why F1 Score?

In healthcare classification, the classes may be imbalanced and both false positives and false negatives can matter. F1 can therefore be more informative than accuracy in some settings.

However, healthcare evaluation should usually consider **recall/sensitivity, specificity, precision, F1, ROC-AUC/PR-AUC, and the clinical costs of errors**, rather than relying on one metric.

---

## Step 9: Evaluate the Model

Make predictions:

```
y_pred = grid_search.predict(X_test)
```

Calculate evaluation metrics:

```
from sklearn.metrics import (  
  
    accuracy_score,  
  
    precision_score,  
  
    recall_score,  
  
    f1_score,  
  
    confusion_matrix  
  
)
```

```
print("Accuracy:",  
      accuracy_score(y_test, y_pred))  
  
print("Precision:",  
      precision_score(y_test, y_pred))  
  
print("Recall:",  
      recall_score(y_test, y_pred))  
  
print("F1 Score:",  
      f1_score(y_test, y_pred))  
  
print("\nConfusion Matrix:")  
  
print(confusion_matrix(y_test, y_pred))
```

---

## Step 10: Interpret the Results

In healthcare, **False Negatives** can be particularly important.

A false negative means:

Patient actually has disease

↓

Model predicts No Disease

This may delay diagnosis or treatment.

Therefore, depending on the medical use case, we may prioritize **high recall/sensitivity** so that fewer patients with the disease are missed.

At the same time, false positives also have costs, such as unnecessary testing or treatment, so the desired operating point should be decided with clinical and business stakeholders.

---

## Business Value of the Healthcare Decision Tree Model

A well-designed model could provide several business benefits:

### 1. Early Risk Identification

The model can help identify patients who may require further medical evaluation.

### 2. Improved Decision Support

It can provide an additional data-driven signal for healthcare professionals.

### 3. Better Resource Allocation

Hospitals can prioritize patients who may need additional tests or attention.

### 4. Reduced Operational Costs

Better risk stratification can help reduce unnecessary testing and improve resource utilization.

### 5. Patient Management

High-risk patients can potentially be followed more closely.

### 6. Explainability

Decision Trees can be easier to explain than many complex Machine Learning models because their predictions can be represented as a sequence of decision rules.

---

## Important Healthcare Considerations

Because this is a healthcare application, model accuracy alone is not sufficient.

The system should also consider:

- Patient privacy
- Data security
- Bias and fairness
- Clinical validation
- False positives and false negatives
- Model interpretability
- Data leakage
- Regular monitoring after deployment

The model should be treated as a **decision-support tool**, not as a replacement for qualified medical judgment.

---

## Complete Healthcare Workflow

Raw Healthcare Data



Data Understanding



Handle Missing Values



Encode Categorical Features



Train-Test Split



Train Decision Tree



Hyperparameter Tuning



Model Evaluation

↓

Interpret Results

↓

Clinical/Business Validation

↓

Deployment & Monitoring

---

## Final Quick Revision

| Topic                          | Key Point                          |
|--------------------------------|------------------------------------|
| Decision Tree                  | Tree-based supervised ML algorithm |
| Classification                 | Predicts categories/classes        |
| Gini Impurity                  | Measures node impurity             |
| Entropy                        | Measures uncertainty               |
| Information Gain               | Reduction in entropy               |
| Pre-Pruning                    | Stops tree growth early            |
| Post-Pruning                   | Removes unnecessary branches       |
| <code>max_depth</code>         | Controls tree depth                |
| <code>min_samples_split</code> | Minimum samples needed to split    |

|                         |                                                 |
|-------------------------|-------------------------------------------------|
| GridSearchCV            | Finds good hyperparameters                      |
| Feature Importance      | Measures feature contribution                   |
| Classification Accuracy | Correct predictions / total predictions         |
| MSE                     | Average squared prediction error                |
| Healthcare Recall       | Important when missing positive cases is costly |

## Key Formulas

### Gini Impurity

$$\text{Gini} = 1 - \sum_i p_i^2$$

### Entropy

$$\text{Entropy} = - \sum_i p_i \log_2(p_i)$$

### Information Gain

$$IG = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$$

Mean Squared Error

$$MSE =$$

$$\frac{1}{n}$$

$$\sum$$

$$(y_i - \hat{y}_i)^2$$

$$y_i$$

$$- \hat{y}_i$$

$$=$$

$$y_i$$

$$\hat{y}_i$$

$$i$$

$$)$$

$$2$$

Final Definition

**Decision Tree is a supervised Machine Learning algorithm that makes a sequence of feature-based decisions to classify observations or predict numerical values.**

---